Formal Verification of the LZ77 and LZ78 Compression Algorithms

Alexander Shekhovtsov Clément Tsedri Ozair Faizan

May 26th 2026

Introduction

- **Goal:** Formally verify the Lempel-Ziv compression algorithms (LZ77 & LZ78).

Problem Statement

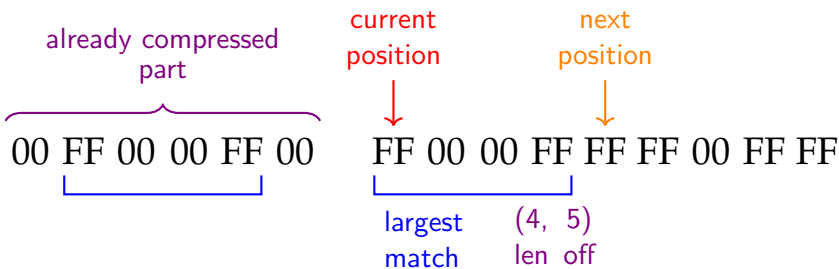
- **End-to-End Correctness:** Prove that decompression exactly reverses compression (i.e., `decompress (compress s) = s`).
- Upper bounds on the worst-case compressed output lengths.
- Initially, focused on low-level C verification of LZ77.

Approach

- We separated the core compression logic from the byte/bit serialization.
- **Pass 1:** Convert the input bytes/bits sequence into an intermediate list of abstract Tokens.
- **Pass 2:** Serialize those Tokens into bytes (LZ77) or bits (LZ78).

LZ77 (LZSS) Algorithm

- Operates over a sliding window, finding the longest match for upcoming bytes.
- Emits Literal tokens (single byte) or Reference tokens (two bytes: 4-bit length and 12-bit offset).



LZ77 Token Representation

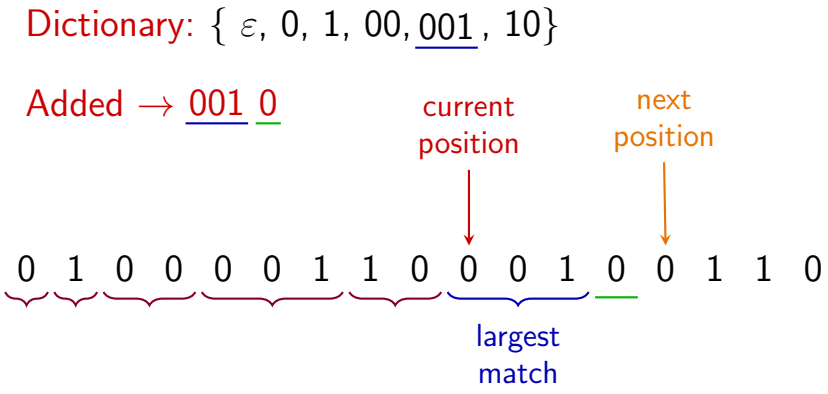
```
Inductive Token :=
  | Lit (b: byte)
  | Ref (length offset: nat). (* 4bit length, 12bit offset *)
```

- `Lit (b: byte)`: the flag bit is set to 1, and the byte `b` emitted.
- `Ref (length offset: nat)`: the flag bit is set to 0, two bytes are emitted:
  - 4 bits for length: $3 \leqslant$ `length` $\leqslant 18$ (bias of $+3$)
  - 12 bits for offset: $3 \leqslant$ `offset` $\leqslant 4098$ (bias of $+3$)

Tokens are processed in chunks of 8, each chunk is prepended with a flag byte indicating the type of each of the 8 following tokens.

LZ78 Algorithm

- Uses a dictionary-based approach, dynamically builds a dictionary of seen bit sequences starting from an empty entry.
- Finds the largest prefix in the dictionary and emits `(index, next_bit)` tokens.



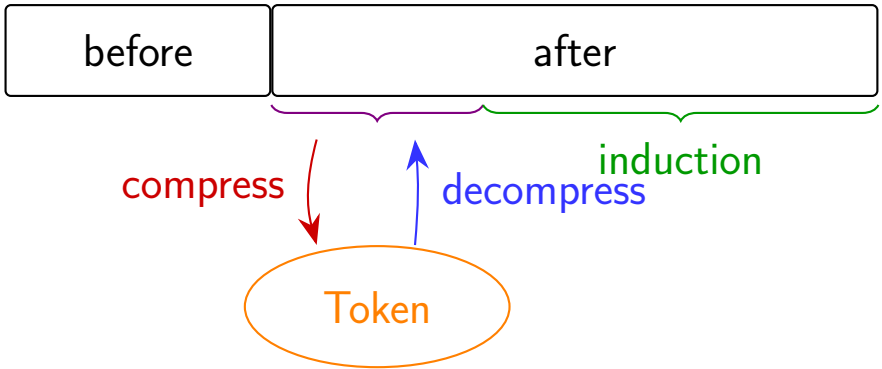
LZ78 Token Representation

```
Inductive Token :=
  | Tok (index: nat) (phr : list bool) (next: bool)
  | Last (index: nat) (phr : list bool).
```

We encode `index` using $\lceil \log_2 |\texttt{dict}| \rceil$ bits so that the detokenizer can decode it. The encoded `index` followed by `next` is emitted.

LZ77 & LZ78 Correctness Theorem Overview

- Explicitly use the history (*before*) and remaining bytes to compress (*after*) in the proof for LZ77.
- Prove by induction on l that $\forall after, |after| \leqslant l$, decompress' *before* (compress' *before after*) = *after*.
- For LZ78, maintain dictionary *dict* instead of *before*.



Upper Bounds

- **LZ77 Bound:**

$$|\texttt{compress\_to\_bytes s}| \leqslant \frac{9 \cdot |s| + 7}{8} + \underbrace{\frac{\log_2 |s|}{7} + 1}_{\text{For length encoding}}.$$

- **LZ78 Bound:** For $|s| \geqslant 4096$

$$|\texttt{compress\_to\_bits s}| \leqslant |s| + 21\frac{|s|}{\log_2 |s|} = |s| + o(|s|).$$

LZ77 Upper Bound Theorem Overview

- Assign a weight to each type of token: 1 for a literal and 3 for a reference
- Prove that the total weight of the compressed string is at most the size of the original string
- Since we have to use an extra flag bit to distinguish between token types, the worst case is then at most 9 bits used per original bytes (since references remove at least 3 bytes and uses 2)
- This gives a bound on the encoded string s, ignoring the length encoding:

$$|\texttt{compress\_to\_bytes s}| \leqslant \frac{9 \cdot |s| + 7}{8}$$

LZ78 Upper Bound Theorem Overview

- LZ78 partitions input string s into m **distinct** phrases $p_1 + p_2 + \ldots + p_m = s$.
- We split the proof of the upper bound $|compress\_to\_bits(s)| \leq |s| + o(|s|)$ into two goals:
- $|compress\_to\_bits(s)| \leq m \cdot (\log_2(m) + 2)$.
- $|s| \geq m \cdot (\log_2 m - 2)$.
- The first bound holds since for each phrase *compress_to_bits* spends $\log_2 m + 1$ bits to encode its position in the dictionary and 1 bit to encode its last element.
- The second bound is equivalent to proving that for any list of distinct strings l, $\sum_i |l_i| \geq l \cdot (\log_2 l - 2)$.

Proof of Combinatorial Theorem Overview

- Prove that there can be at most $2^{k+1} - 1$ distinct strings of length $\leq k$.
- For a list of distinct strings l, prove that there are at least $|l| + 1 - 2^k$ strings of length $\geq k$.
- Express the sum of lengths of all strings $\sum_i |l_i|$ as

$$\sum_i \sum_k I(|l_i| \geq k).$$

- Prove that summations can be exchanged and use established lower bound

$$\sum_i \sum_k I(|l_i| \geq k) = \sum_k \sum_i I(|l_i| \geq k) \geq \sum_{k \leq \log_2 |l|} \left(|l| + 1 - 2^k \right) \geq |l| \cdot (\log_2 |l| - 2).$$

Verified Software Toolchain

- **Initial Plan:** Verify low-level C implementations using VST (Verified Software Toolchain).
- **Challenge:** Verifying low-level memory operations and pointer arithmetic in VST was excessively time-consuming.
- **Things that we learnt:** It is advantageous to use recursion when writing C implementation rather than loops since you can use specification of your function as a guarantee to what recursive call returns instead of writing a new one for a for-loop.

Future Work

- **VST Equivalence:** Define a VST-friendly C and Rocq implementations and prove their equivalence.

Conclusion

- We have implemented two popular compression algorithms, LZ77 and LZ78 in Rocq.
- We have successfully proved their correctness as well as interesting upper bounds on compressed sizes.
- We have experienced a different verification framework VST which turned out to be more challenging than anticipated.
- This however taught us to recognise when it is more productive to switch focus to a more feasible verification task.